

“Event Management Mobile Application for Attendee Self-Service”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

P.Praveena

2520090057

P.Srija

2520090053

D.Varun Teja Reddy

2520090221

Section 12 Team 23 CS-IT

Under the Esteemed Guidance of

Kaja Shareef

Head of Department

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “Event Management Mobile Application for Attendee Self-Service” is a record of bonafide work of P.Praveena - 2520090057, P.Srija - 2520090053 and D.Varun Teja Reddy - 2520090221 submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 23, during 2026-2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

P.Praveena - 2520090057

P.Srija - 2520090053

D.Varun Teja Reddy - 2520090221

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled “Event Management Mobile Application for Attendee Self-Service” is a bonafide work done and submitted by P.Praveena - 2520090057, P.Srija - 2520090053 and D.Varun Teja Reddy - 2520090221 in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 23, at KLH University during 2026-2027, Trimester 4.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation goes to our Course Coordinator and project guide Kaja Shareef, Department of Computer Science and Information Technology, for their support, encouragement and guidance throughout this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the *Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

P.Praveena - 2520090057

P.Srija - 2520090053

D.Varun Teja Reddy - 2520090221

Abstract

The Event Management Mobile Application for Attendee Self-Service is a web-based application designed to help attendees independently discover events, reserve seats and manage their participation. Manual registration queues, paper tickets and separate payment records can cause delays and confusion. College festivals, cultural performances, hackathons and workshops need a clear process for registration, payment, confirmation and entry verification.

The proposed system provides a responsive interface for account registration, login, event search, category filtering, seat booking and digital QR tickets. Attendees can view schedules, download tickets, manage bookings and request eligible cancellations. Administrators can manage events, monitor bookings, verify QR check-ins, review feedback and publish announcements.

The application uses React with Vite for the frontend, Node.js and Express for the backend, and MySQL for storing related records. The documented database has eleven tables covering users, categories, events, seat reservations, bookings, booking items, payments, tickets, refunds, feedback and notifications. Transactions and row locks support consistent seat allocation. Razorpay test-mode payments are verified on the backend before paid bookings are confirmed. Unique QR tickets support entry checks and duplicate-entry prevention. The project aims to reduce manual coordination and give attendees a convenient self-service journey. The supplied project documentation describes these features; execution outcomes must be confirmed during the project demonstration.

Index

S.No.	Chapters	Topics	Page No.
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	8
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	9
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	10
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	12
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	13
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	15
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	17
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	18

S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	19
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	20
13	References	– Documentation, tools, and resources used	21
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Default Login Credentials – API Reference Summary – Screenshots (to be added by the student)	22

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Colleges organize cultural festivals, dance and music programmes, workshops, seminars and hackathons throughout the year. Attendees often depend on separate registration forms, manual payment checking and paper tickets. Event details may be spread across messages and posters, making it difficult to find the correct venue, schedule, ticket price and availability. During popular events, queues and unclear booking status can create extra work for organizers and inconvenience for attendees.

The Event Management Mobile Application for Attendee Self-Service brings these steps into one database-backed application. Attendees can browse events, reserve seats and manage their tickets. Organizers can manage event information and use digital records to monitor participation. The interface is designed for browser access on mobile phones and computers.

1.2 Problem Statement

The project addresses the difficulty of managing event discovery, attendee registration, ticket booking, payment confirmation and entry verification through disconnected manual processes. The system must show available seats, protect user accounts, keep booking and payment records consistent, and prevent the same QR ticket from being accepted more than once. Attendees need clear confirmation and access to their own bookings without repeatedly contacting an administrator.

1.3 Scope of the Project

The scope includes attendee registration and login, event browsing and filtering, event details and schedules, temporary seat reservations, free and paid booking flows, digital QR tickets, booking history, eligible cancellation and refund tracking. The administrator area supports event management, attendee and booking records, QR check-in, feedback and announcements. React, Express and MySQL form the current web implementation. A dedicated native mobile app and production payment settlement are future extensions.

1.4 Objectives

- Show event details, schedules, ticket prices and available seats clearly.
- Support secure account registration and login with individual email addresses.
- Keep seat reservations, payments and confirmed bookings consistent through database transactions and validation.
- Give administrators a dashboard to manage events, attendees, bookings and entry verification.
- Generate digital QR tickets for confirmed bookings and reject repeated check-in attempts for the same ticket.

- Secure the system with role-based access, so student accounts can never reach admin-only functionality.

1.5 Importance of the Application

By centralizing event, attendee and booking data, the application reduces manual registration work and improves visibility of ticket status. Attendees can discover events and manage their participation from a phone or computer. Organizers gain access to booking records and QR-based entry verification. A clear confirmation screen helps users understand when payment verification and booking confirmation have completed.

1.6 Target Users / Audience

The system serves attendees and administrators. Attendees register with their own email address, browse cultural and technical events, book seats, obtain QR tickets and view their booking history. Administrators manage categories and event information, monitor participation and payments, scan tickets at entry, and review feedback and notifications.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

A development computer with a dual-core processor, at least 4 GB of RAM and sufficient free storage for Node.js dependencies, MySQL and the project files can run the application locally. An 8 GB RAM system provides a more comfortable development environment. Attendees need a modern browser on a phone or computer. QR check-in requires a camera-enabled device or manual ticket-code entry.

2.2 Software Requirements

- Node.js 18 or later, with npm
- MySQL 8.0 or later
- A modern web browser (Chrome, Edge, or Firefox)
- Git, for version control
- MySQL Workbench and Visual Studio Code, for development (optional but recommended)

2.3 Development Tools and Frameworks

React 18 with Vite supports the frontend. Node.js and Express provide the REST API. MySQL 8 or MariaDB stores relational data through mysql2/promise. JWT and bcryptjs support authentication, qrcode generates digital passes, and the Razorpay SDK supports test-mode checkout. VS Code and MySQL Workbench can be used for development; Postman supports API validation.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend uses React 18 with Vite and reusable components for event cards, navigation, event details, booking forms and ticket displays. Lucide React provides interface icons. The attendee journey includes event discovery, seat selection, payment or free confirmation, a success page and My Bookings. Responsive styling helps the same interface work on phones and computers.

3.2 Backend

The Node.js and Express backend exposes REST endpoints for authentication, events, reservations, bookings, payments, tickets, feedback and notifications. Routes receive requests, middleware validates access, and controller or service logic applies business rules. The mysql2/promise connection pool provides database access and supports multi-step transactions.

3.3 Database

The event_management MySQL database contains eleven related tables: users, categories, events, seat_reservations, bookings, booking_items, payments, tickets, refunds, feedback and notifications. Primary keys identify records and foreign keys link related data. Unique email addresses, payment identifiers and ticket codes help prevent duplicate records. Capacity and status checks protect booking consistency.

3.4 Version Control

Git and GitHub can be used to track frontend, backend and database changes in a shared project repository. Environment secrets are kept outside source control.

3.5 Authentication and Security Libraries

The documented backend hashes passwords with bcryptjs and uses JWT for protected requests. Public registration creates attendee accounts. Administrator privileges are assigned through the setup process. Protected routes verify the token and role. Payment signatures are checked on the server before paid tickets are issued; a browser success message alone must not confirm a paid booking.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a standard three-tier architecture, cleanly separating the user interface, the business logic, and the persisted data. Figure 4.1 shows the three layers and how a request flows between them.

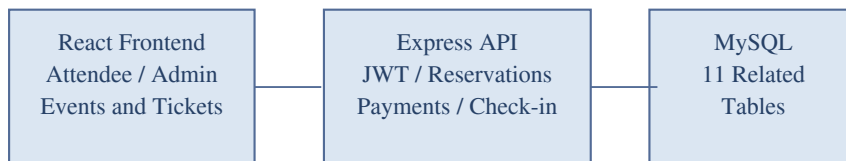


Figure 4.1: Event Management three-tier architecture

4.2 Description of Each Layer

The Presentation Layer is the React single-page application. It never talks to MySQL directly; every piece of data it shows comes from calling the REST API over HTTP. During development, Vite's built-in proxy forwards any request the frontend makes to “/api/...” straight through to the Express server, so the browser never needs to know the backend's actual port or address.

The Application Layer is the Express REST API. It validates authentication, roles, event availability, seat quantities and booking ownership. It creates payment orders and verifies gateway signatures. Database transactions connect reservation updates with booking, payment and ticket records so that a failed operation does not leave partial booking data.

The Data Layer is the MySQL database, accessed through a connection pool (via the mysql2 driver) rather than a single long-lived connection, so multiple requests can be served concurrently without waiting on each other unnecessarily.

4.3 Deployment Architecture

For local development, the Express backend listens on port 5001, the Vite frontend on port 5173, and MySQL on port 3306. The frontend sends API requests to the backend rather than connecting directly to the database. Test-mode gateway credentials are configured on the server. A hosted deployment would add HTTPS, managed secrets, a production database and monitoring.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The schema was normalized to remove redundant data: each fact is stored in exactly one place, and every relationship between entities is expressed through a foreign key rather than by duplicating data across tables. Every table enforces its own invariants directly in the database — not just in application code — using PRIMARY KEY, FOREIGN KEY, UNIQUE, and CHECK constraints, so that invalid data (a negative capacity, an end time before a start time, a duplicate registration) is rejected at the database level even if a bug elsewhere in the application tried to insert it.

5.2 ER Diagram / Database Schema

The database contains eleven related tables; Figure 5.1 shows the main relationships.

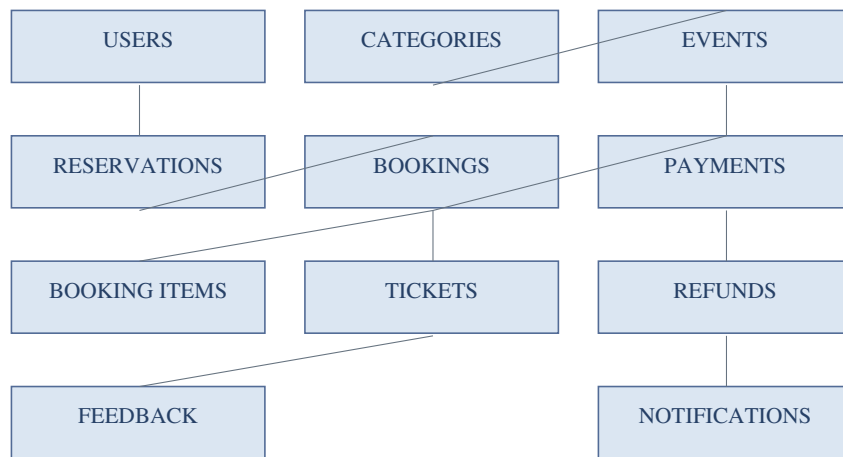


Figure 5.1: Main relationships of the 11-table database

USERS stores attendee and administrator accounts. CATEGORIES groups cultural and technical events. EVENTS stores titles, venue details, dates, prices, capacity and available seats. SEAT_RESERVATIONS holds temporary checkout reservations. BOOKINGS records the attendee, event, seat quantity, total and status, while BOOKING_ITEMS stores individual ticket details. PAYMENTS records gateway orders, payment references and verification status. TICKETS stores unique QR passes and check-in status. REFUNDS tracks eligible cancellation refunds. FEEDBACK stores event ratings and reviews. NOTIFICATIONS stores attendee alerts and announcements. These records are linked through identifiers instead of repeatedly storing the same information.

5.3 Design Decisions and Assumptions

The main design decisions concern temporary seat reservations, payment verification and confirmed ticket issuance. These rules keep the attendee journey and database records consistent.

- Reservation and booking lifecycle: temporary reservations are separate from confirmed bookings.
A seat_reservations record holds seats during checkout for the documented ten-minute period. Failed or abandoned payments release expired reservations. A confirmed booking is created only after the free-booking flow or successful server-side payment verification. Transactions keep seat counts and booking records consistent.
- Payment and ticket integrity: paid event tickets require verified gateway payment records.
Amounts are calculated from database prices. Unique payment references and ticket codes prevent repeated processing. A ticket already checked in must be rejected on another scan.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The application is divided into authentication, event discovery, seat reservations, bookings, payments, tickets, refunds, feedback and notifications. During checkout, the backend checks event availability and reserves the requested quantity. The documented implementation uses `SELECT ... FOR UPDATE` to protect seat counts during concurrent requests. Paid events create a gateway order; the server verifies the returned signature before confirming the booking. Free events can confirm without payment. The confirmed booking is linked to payment and digital-ticket records.

The key consistency rule is that a request can reserve seats only when the available quantity is sufficient. Related updates run in a transaction and must roll back if an operation fails. Confirmation and check-in should be idempotent: repeating a payment response must not create another booking, and rescanning a checked-in ticket must return a conflict instead of allowing entry again.

6.2 Front-end Logic

React state manages the selected event, seat quantity, checkout progress and booking information. Event cards show posters, categories, venues, prices and availability. Clicking an event opens its details and booking options. After free confirmation or verified payment, the application opens a success screen with the booking reference and QR ticket. My Bookings provides booking status, ticket download and eligible cancellation controls. Loading, empty and error states help users understand pending requests.

6.3 API Integration

The frontend sends JSON requests to the Express API at `http://localhost:5001/api`. Protected requests include the JWT in the Authorization header. The server returns event data, reservation details, booking status and ticket information. The interface displays validation errors and redirects to confirmation only after the backend reports a successful booking.

6.4 Authentication and Authorization

Attendees register with their own name, email and password. Login checks the stored password hash and returns a signed token. Public sign-up must not allow an administrator role. Admin accounts are configured through the setup command. Backend middleware verifies JWT access and checks roles; ownership checks ensure attendees can view or cancel only their own bookings.

CHAPTER 7: FEATURES

7.1 List of Main Features

Attendee-facing features:

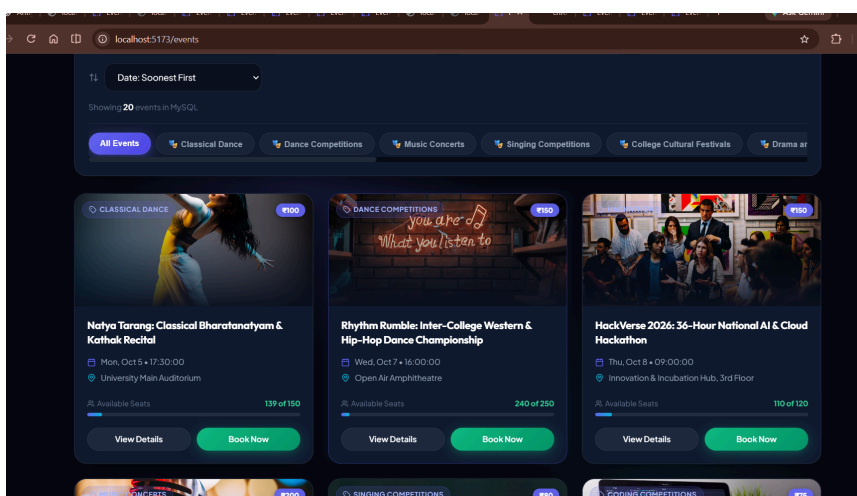
- Browse, search, and filter upcoming events by name and type
- View full event details, including venue, date, schedule, price and available seats
- Reserve seats and complete free or paid ticket booking
- View My Bookings with confirmed, cancelled and payment status information
- Download digital QR tickets and manage eligible cancellations
- View a personal profile

Administrator-facing features:

- Add and edit event categories, descriptions, posters, schedules and ticket prices with validation for capacity and availability
- Create and manage cultural programmes, hackathons, workshops and competitions
- View dashboard summaries of events, bookings, attendees and payment records and review participation information
- View and manage all bookings, including cancellation
- View attendee records and booking details by event
- Scan QR tickets, record check-in timestamps and reject repeated entry attempts using ticket status validation

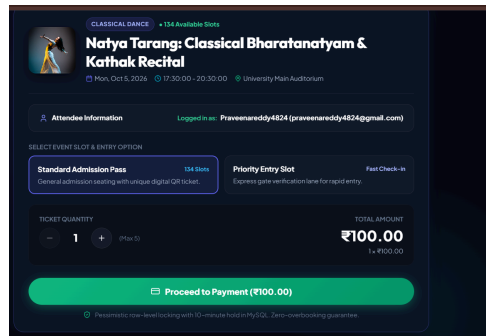
7.2 How Each Feature Works

Event booking: the attendee browses the event catalogue, opens an event and selects a ticket quantity. The backend checks availability and creates a temporary reservation. Free events can confirm directly; paid events require gateway checkout and server verification. The confirmation page shows the booking reference and digital ticket. The supplied catalogue screenshot below illustrates event discovery.



Payment confirmation: the application confirms a paid booking only after the backend verifies the payment signature and updates the related booking records.

The success screen should show the booking ID, payment reference and downloadable QR pass. The supplied checkout screenshot below shows the payment interface.



QR ticket check-in: the administrator scans the digital ticket or enters its code. The backend checks that the ticket is active and belongs to a valid booking. A successful scan records entry. A cancelled ticket or one that is already checked in is rejected. The same ticket must never authorize repeated entry.

Screenshot to be added from the running application.

CHAPTER 8: TESTING

8.1 Postman API Testing

Postman can validate the authentication, event, reservation, booking, payment and ticket endpoints. Login returns a JWT that is supplied to protected requests. A complete test flow creates an attendee account, selects an event, reserves seats, confirms a free or paid booking and checks the resulting ticket. Invalid input and unauthorized requests should also be tested.

8.2 Integration Testing

Integration testing should verify the full attendee journey against MySQL: registration, login, browsing, reservation, payment verification, confirmation, ticket download, cancellation and check-in. Database records should be checked after each step. Concurrent reservation requests and repeated payment callbacks should also be tested. The supplied README includes `npm run test:flow`, but no execution log was provided for this report.

8.3 Tools Used for Testing

- Postman — manual and scripted API testing
- curl (command line) — scripted regression testing across every phase of development
- MySQL command-line client — direct verification of table contents and constraint behavior
- Manual browser testing — verifying the React frontend's behavior and appearance

8.4 Test Cases and Results

The following cases describe the expected validation results. Actual results are marked Pending until recorded during execution of this project.

#	Test Case	Expected Result	Actual Result
1	Login with wrong password	401; login rejected	Pending
2	Attendee calls admin-only route	403 Forbidden	Pending
3	Request seats beyond availability	Rejected; no overbooking	Pending
4	Paid booking without verification	Rejected; no ticket issued	Pending
5	Book a free event	Confirmed with QR ticket	Pending
6	Verify an invalid payment signature	Rejected; booking unconfirmed	Pending
7	Repeat a verified payment response	No duplicate booking	Pending

#	Test Case	Expected Result	Actual Result
8	Request a negative seat quantity	400; invalid input rejected	Pending
9	Scan a valid active QR ticket	Check-in recorded once	Pending
10	Scan the same QR ticket again	409; duplicate entry rejected	Pending
11	Cancel an eligible booking	Cancelled; seats restored	Pending
12	Scan a cancelled ticket	Entry rejected	Pending
13	Open another attendee booking	Access denied	Pending
14	Abandon reserved-seat checkout	Expired seats released	Pending

Payment-flow validation is especially important. A static UPI QR image cannot tell the browser that a payment has completed. The documented gateway flow verifies the payment on the backend before opening the confirmation page. Testing should confirm that cancelled or invalid payments do not issue tickets and that valid confirmation occurs only once.

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application is deployed locally for development and demonstration as follows:

- Load the database: `mysql -u root -p < database/schema.sql`
- Start the backend: `cd backend, npm install, npm start (port 5001)`
- Start the frontend: `cd frontend, npm install, npm run dev (port 5173)`
- Open `http://localhost:5173` and register with your own email and password

9.2 Environment Configuration

Configure `backend/.env` with `PORT=5001`, database host and port, `DB_USER`, `DB_PASSWORD` and `DB_NAME=event_management`. Set a private `JWT_SECRET` and the Razorpay test credentials. The documented setup uses `ADMIN_NAME`, `ADMIN_EMAIL` and `ADMIN_PASSWORD` for the administrator account. Run `npm run seed` and `npm run setup:admin` as needed. Keep actual secrets outside source control.

9.3 Hosting of Frontend and Backend

The project is documented for local demonstration using a Vite frontend, Express backend and MySQL database. A production deployment requires a compiled frontend, an HTTPS API, protected environment configuration and database backups. The API base URL and CORS settings must match the hosted frontend. The project should use gateway test mode during demonstrations.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Payment confirmation: displaying a payment QR does not automatically verify that money was paid. The booking page must receive a verified backend result before issuing a ticket or showing success.
- Concurrent seat booking: simultaneous requests can compete for the last available tickets. Reservation expiry, database row locks and transactions are needed to keep seat counts consistent.
- QR ticket reuse: the same pass could otherwise be scanned repeatedly at entry. Ticket status and check-in timestamps must be checked on the server.

10.2 Solutions Applied

The documented payment approach uses Razorpay test checkout and HMAC-SHA256 signature verification on the server. Booking amounts come from database prices. Paid tickets are issued only after the server confirms the payment. A static UPI payment image requires a separate verified confirmation mechanism.

The seat-reservation approach holds seats temporarily and uses row-level locking inside a database transaction. Expired reservations release seats back to the event pool. Cancellation updates the booking and available-seat count together. Repeated gateway responses should return the existing result instead of creating another booking.

The QR entry workflow records check-in against a unique ticket code and rejects tickets that are already checked in or cancelled. These measures must be validated with the full-flow test and administrator demonstration.

10.3 Current Limitations or Known Bugs

- Production payments and external refunds require gateway configuration and verification.
- Public sign-up must remain limited to attendees; administrators use a controlled setup process rather than choosing an admin role during registration.
- Native Android or iOS distribution is outside the current React web implementation. The responsive browser interface supports mobile access.
- Full execution evidence and cloud-deployment results were not supplied with the project files and should be recorded before final evaluation.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Email-based password reset and account verification.
- A safe, audited admin UI for promoting an existing user to an admin role, rather than requiring direct database access.
- A calendar view of events and personal schedules, helping attendees plan their participation across multiple programmes.
- Email or SMS notifications for registration confirmation and event reminders.
- Offline-capable QR scanning with controlled synchronization and detailed check-in audit records for larger events.

11.2 Possible Integrations or Optimizations

- Calendar integration to save confirmed event dates and schedules to attendee devices automatically.
- Caching popular event listings and optimizing database queries during peak ticket-booking periods.
- Containerizing the application with Docker for a consistent, reproducible deployment environment.
- Expanding automated API, payment and concurrency tests to catch regressions before deployment and adding monitoring for booking and payment failures.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

The Event Management Mobile Application for Attendee Self-Service provides a database-driven approach to event participation. The documented application combines a React frontend, an Express API and an eleven-table MySQL database to support event discovery, booking, payment records and digital ticket management. The responsive web interface gives attendees access through phones and computers.

12.2 What Was Achieved

The project documentation covers individual user registration, cultural and technical event browsing, seat reservations, free and paid booking flows, backend payment verification, digital QR passes and attendee booking history. Administrator functions include event management and ticket check-in. The supplied screenshots illustrate event discovery and checkout. The validation plan records expected results; final execution evidence should be added after testing the running application.

12.3 Skills Learned

The project develops practical skills in relational database design, frontend component development, REST API integration, authentication and role checks. It also demonstrates why transactions and row locks matter when multiple attendees reserve seats. Payment verification and QR status checks connect user-interface actions to trusted backend records. Working as Team 23 helped the members coordinate implementation, documentation and demonstration tasks.

CHAPTER 13: REFERENCES

- React — Official Documentation, <https://react.dev>
- Vite — Official Documentation, <https://vitejs.dev>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- Express.js — Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- mysql2 (Node.js MySQL driver) — npm documentation, <https://www.npmjs.com/package/mysql2>
- jsonwebtoken — npm documentation, <https://www.npmjs.com/package/jsonwebtoken>
- bcryptjs - npm documentation, <https://www.npmjs.com/package/bcryptjs>
- Postman Learning Center, <https://learning.postman.com>
- MDN Web Docs — JavaScript and Web APIs reference, <https://developer.mozilla.org>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

Illustrative transaction for reserving available event seats:

```
START TRANSACTION;
SELECT available_seats FROM events WHERE id = ? FOR UPDATE;
-- Check quantity and availability before updating.
UPDATE events SET available_seats = available_seats - ?
WHERE id = ? AND available_seats >= ?;
-- Write a temporary seat reservation in this transaction.
COMMIT;
```

Appendix B: Installation and Setup Instructions

- `mysql -u root -p < database/schema.sql`
- `cd backend && npm run seed && npm run setup:admin`
- `cd backend && npm install && npm start (backend, port 5001)`
- `cd frontend && npm install && npm run dev (frontend, port 5173)`
- Open `http://localhost:5173` in a browser

Appendix C: Default Login Credentials

Attendees use their own registered credentials; no fixed student login is required.

Role	Email
Admin	Configured through backend/.env and <code>npm run setup:admin</code>
Attendee	Register using your own name and email
Password	Chosen during registration and stored as a hash
Test gateway	Use valid Razorpay test credentials configured on the server

Appendix D: API Reference Summary

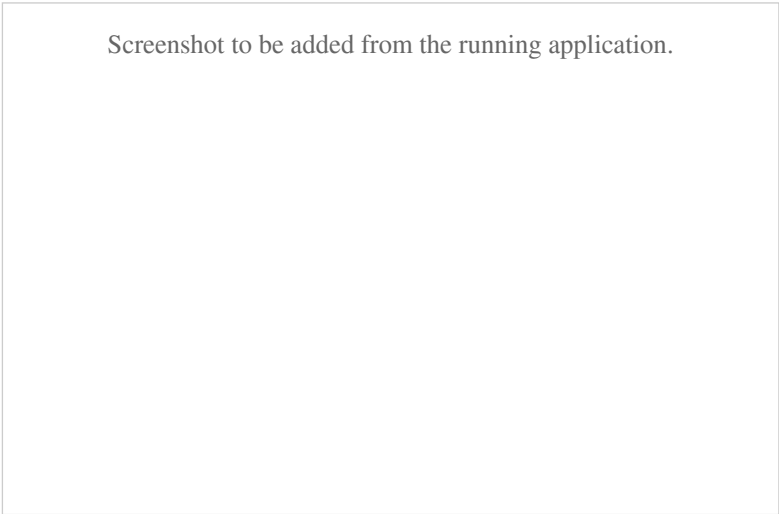
API resources cover authentication, events, seat reservations, bookings, payments, tickets and attendee self-service. The backend uses the `/api` prefix.

Resource	Endpoints
Auth	Register, login and current-user access
Events	List events, details and administrator event management
Reservations	Create temporary reservations and check availability
Payments	Create gateway orders and verify payment responses
Bookings	Confirm, list and cancel owned bookings
Tickets	Retrieve QR tickets and perform administrator check-in

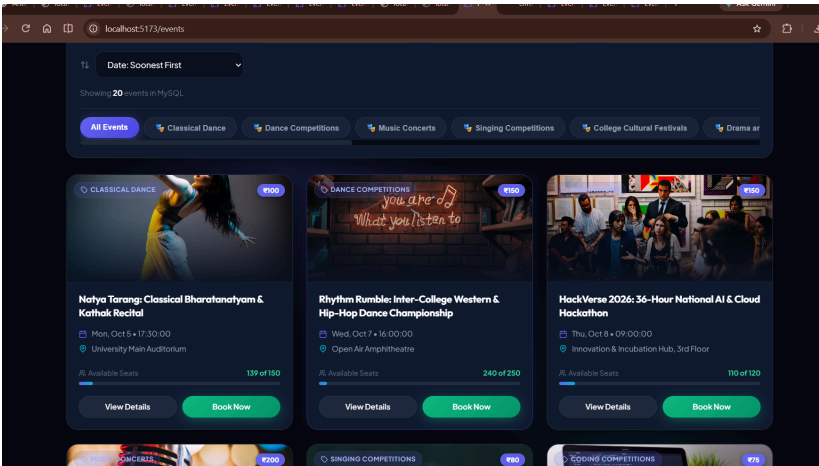
Appendix E: Screenshots

The screenshots below relate to the supplied attendee application. Sections without a supplied screenshot retain the template space for the corresponding project output.

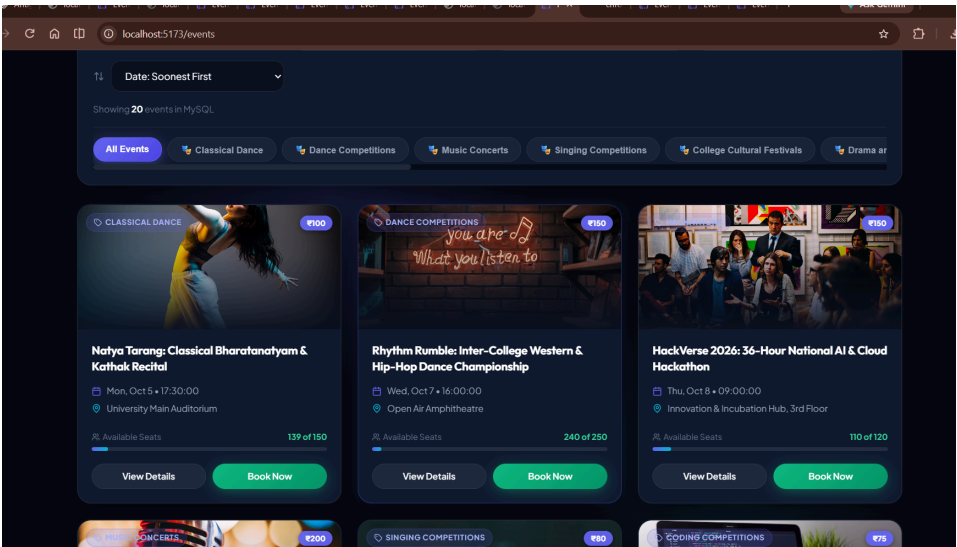
1. Login Page



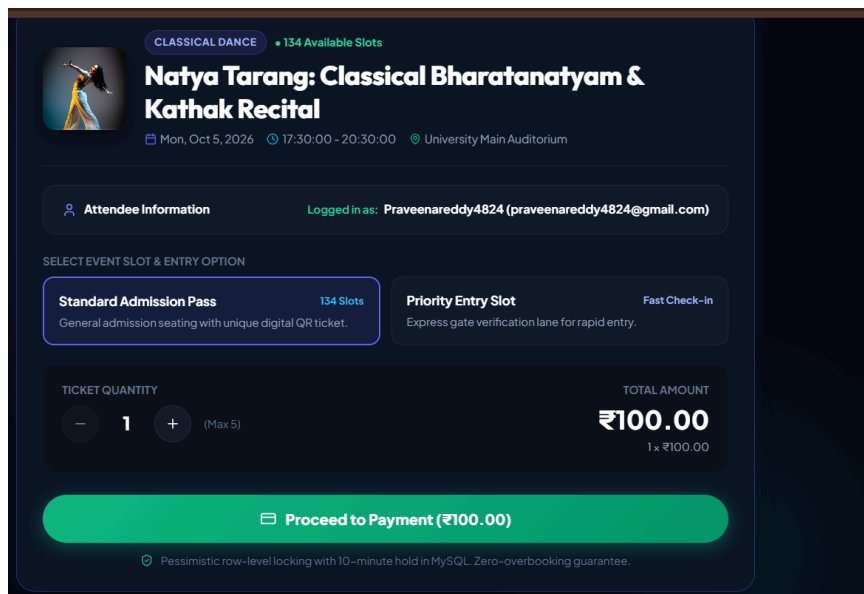
2. Attendee Event Dashboard



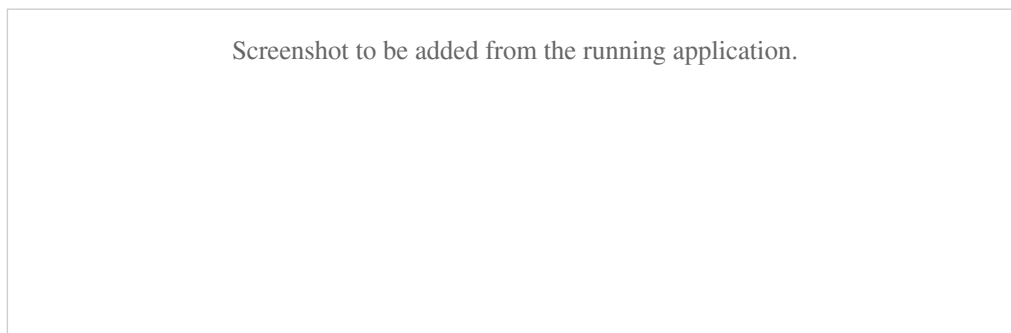
3. Events Page (attendee view)



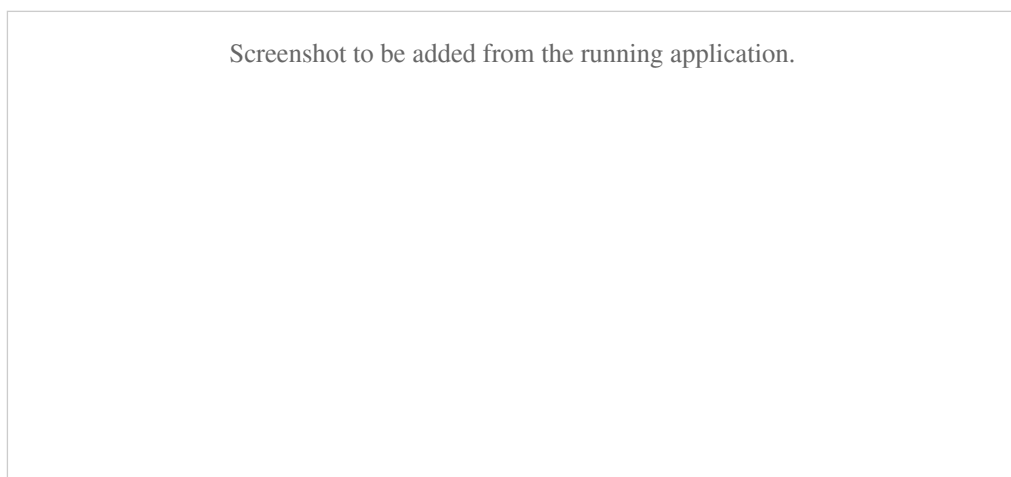
4. Event Booking and Payment Page



5. Admin Dashboard



6. Manage Events / Categories Page



7. Booking Confirmation and Digital QR Ticket

Screenshot to be added from the running application.

8. Administrator QR Check-in Page

Screenshot to be added from the running application.